TD6 – Système de RAG pour la question-réponse sur les films

BELGUEDJ Nassilya
BAKHOUCHE Rachel
BOUACHE Katia
AMMOUR Nadia
FONHOUE DASSY Lionel Bryant

15 octobre 2026

1 Contexte et objectif

Un système RAG répond à une question en deux temps : il **retrouve** (retrieval) les passages pertinents dans une base documentaire, puis **génère** (reply) une réponse à partir de ces passages. L'objectif du TD est de maximiser la qualité de ces deux étapes, mesurée respectivement par le MRR et la similarité sémantique des réponses.

Les données fournies sont : un ensemble de pages Wikipédia de films au format Markdown, et un jeu de questions associé à chaque question le passage censé contenir la réponse (`text_answering`) et la réponse attendue (`expected_reply`). Les expérimentations de ce rapport portent sur les 5 films de référence (*Inception*, *The Dark Knight*, *Deadpool*, *Fight Club*, *Pulp Fiction*).

2 Métrique d'évaluation

La qualité du retrieval est mesurée par le **Mean Reciprocal Rank**. Pour chaque question, on note le rang r auquel apparaît le premier chunk contenant le passage de réponse ; le score vaut $1/r$ (et 0 si aucun chunk récupéré ne le contient). Le MRR est la moyenne de ces scores :

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rang}_q}.$$

Un MRR proche de 1 signifie que le bon passage est systématiquement classé en tête. Cette métrique a une conséquence importante pour le découpage : comme la correspondance se fait par *sous-chaîne exacte*, un passage coupé entre deux chunks devient introuvable.

3 Architecture et méthodes explorées

3.1 Correction préalable : ordre du classement

Le code initial sélectionnait les 5 meilleurs chunks mais les renvoyait dans l'ordre croissant de similarité, plaçant donc le *meilleur* chunk en dernière position. Comme l'évaluation calcule le rang selon la position dans la liste, le MRR était artificiellement dégradé. La correction (tri décroissant, meilleur chunk en premier) constitue le véritable point de départ des mesures.

3.2 Découpage contextuel et chevauchement

Le découpage d'origine ne conservait que le contenu des sections Markdown. J'ai modifié `chunk_markdown` pour (i) préfixer chaque chunk par sa hiérarchie de titres (dont le titre du film), afin de conserver le contexte, et (ii) introduire un chevauchement (*overlap*) entre chunks voisins, pour éviter de couper un passage de réponse en deux. Le préfixe étant ajouté en tête, il ne perturbe pas la correspondance par sous-chaîne.

3.3 Instruction de requête pour l'embedder

L'embedder `BAAI/bge-base-en-v1.5` attend une instruction spécifique placée devant les *questions* (et non devant les passages). Cette instruction n'était jamais appliquée dans le code initial. Je la préfixe désormais manuellement au texte de chaque question avant l'encodage.

3.4 Reranking par cross-encoder

La recherche par embeddings (*bi-encoder*) encode question et passage séparément : rapide, mais grossière. Un **cross-encoder** (`BAAI/bge-reranker-base`) relit en revanche chaque paire (question, passage) conjointement et produit un score de pertinence fin. Le pipeline devient donc : présélection large par embeddings (*pool* de 30 chunks), puis reclassement par le reranker, et conservation des 5 meilleurs.

3.5 Recherche hybride BM25 + dense

BM25 est un algorithme de recherche *lexicale* : il favorise les chunks partageant les mots exacts de la question, notamment les termes rares (noms propres, dates). Il complète la recherche sémantique, qui comprend le sens mais est parfois imprécise sur les mots exacts. Les deux classements sont combinés par **Reciprocal Rank Fusion** (RRF) : chaque chunk reçoit un score $\sum_i 1/(k + r_i)$ où r_i est son rang dans le classement i et $k = 60$.

4 Détails d'implémentation

Le code est organisé dans `src_rag/` : `models.py` (classe `RAG`, découpage, recherche), `embedding.py` (encodage avec cache disque) et `evaluate.py` (évaluation et traçage MLflow). Les paramètres (`chunk_size`, `overlap`, `use_reranker`, `use_bm25`, etc.) sont passés par configuration, ce qui permet à MLflow de les tracer automatiquement. Un cache `joblib` mémorise les paires (texte, config) $\rightarrow$ embedding sur disque, évitant de réencoder les chunks déjà vus entre deux runs.

5 Résultats

5.1 Effet de la taille de chunk (avec reranker)

Taille de chunk (tokens)	MRR
128	0,354
192	0,398
256	0,426
384	0,449
512	**0,473**

TABLE 1 – MRR en fonction de la taille de chunk (overlap = 32, reranker activé).

Contre-intuitivement, le MRR croît avec la taille des chunks. L'explication tient à la métrique : des chunks plus petits coupent plus souvent le passage de réponse entre deux morceaux, le rendant introuvable. La valeur 512 constitue par ailleurs la limite d'entrée de l'embedder bge-base ; au-delà, le texte serait tronqué.

5.2 Comparaison des méthodes de recherche (chunk = 512)

Méthode	MRR
Dense seul	0,394
Hybride (dense + BM25) seul	0,436
Dense + reranker	**0,473**
Hybride + reranker	0,467

TABLE 2 – MRR selon la méthode de recherche (overlap = 32).

Deux enseignements. D'abord, **BM25 apporte un gain net en l'absence de reranker** (0,394 → 0,436, soit +10 %) : le signal lexical remonte des passages mal classés par le sémantique. Ensuite, **BM25 devient redondant lorsque le reranker est présent** : ce dernier récupère déjà ces passages, et l'hybride introduit alors une légère dilution (−1 %). Les deux techniques visant le même objectif, les cumuler n'est pas rentable sur ce jeu de données.

5.3 Effet de la taille du pool du reranker (chunk = 256)

Pool	MRR
30	0,4263
50	0,4277

TABLE 3 – Effet de l'élargissement de la présélection envoyée au reranker.

Élargir la présélection de 30 à 50 n'apporte quasiment rien (+0,001) : le bon chunk est presque toujours déjà présent dans les 30 premiers. La valeur 30 est conservée (gain négligeable, calcul plus rapide).

5.4 Génération de réponse (reply similarity)

La similarité sémantique entre la réponse générée et la réponse attendue est mesurée par cosinus d'embeddings (seuil de correction fixé à 0,7). Résultats pour la configuration retenue :

Configuration	reply_similarity	% correct
Config retenue (512 + reranker)	0,777	55,6 %

TABLE 4 – Qualité de génération sur l'échantillon de 9 questions (seuil de correction = 0,7).

La similarité moyenne des réponses (0,777) est élevée, mais le taux de réponses jugées correctes reste modéré (55,6 %, soit 5 sur 9). L'écart s'explique par le seuil binaire à 0,7 : des réponses sémantiquement proches mais formulées différemment sont comptées comme incorrectes. Le petit effectif (9 questions) rend par ailleurs cette mesure sensible, chaque question pesant environ 11 %.

6 Configuration retenue

La meilleure configuration sur les 5 films est :

```
chunk_size = 512, overlap = 32, add_headers = True, use_reranker = True
```

avec un MRR de **0**,**473**, contre 0,352 pour la recherche dense initiale, soit un gain de +34 %.

7 Discussion et limites

Les expérimentations ont été menées sur 5 films, ce qui limite la portée des conclusions. Plusieurs résultats seraient probablement différents à l'échelle des 100 films : le découpage contextuel (préfixe de titre) et BM25 devraient y gagner en utilité, car ils aident à discriminer entre documents nombreux et proches. La limite d'entrée de l'embedder (512 tokens) plafonne par ailleurs le gain obtenu via la taille de chunk ; un embedder à contexte plus long pourrait repousser cette limite

8 Conclusion

À partir du code fourni, la correction d'un défaut de classement puis l'ajout d'un reranking par cross-encoder ont permis d'améliorer sensiblement la qualité du retrieval (+34 % de MRR). Le tuning de la taille de chunk et l'étude de la recherche hybride ont précisé les compromis en jeu, notamment la redondance entre BM25 et reranking. L'ensemble du pipeline est paramétrable et tracé, facilitant la reproduction et l'extension des expériences.